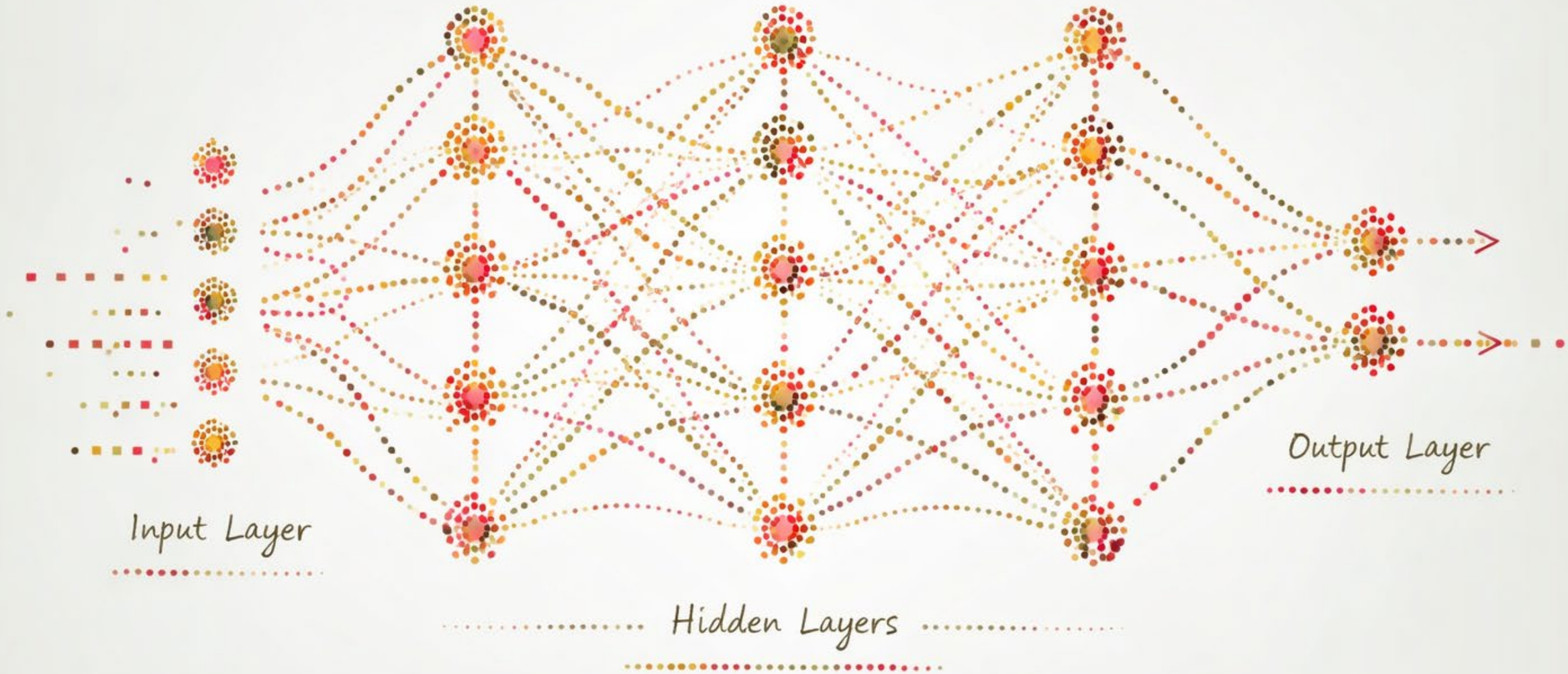


ARTIFICIAL NEURAL NETWORKS

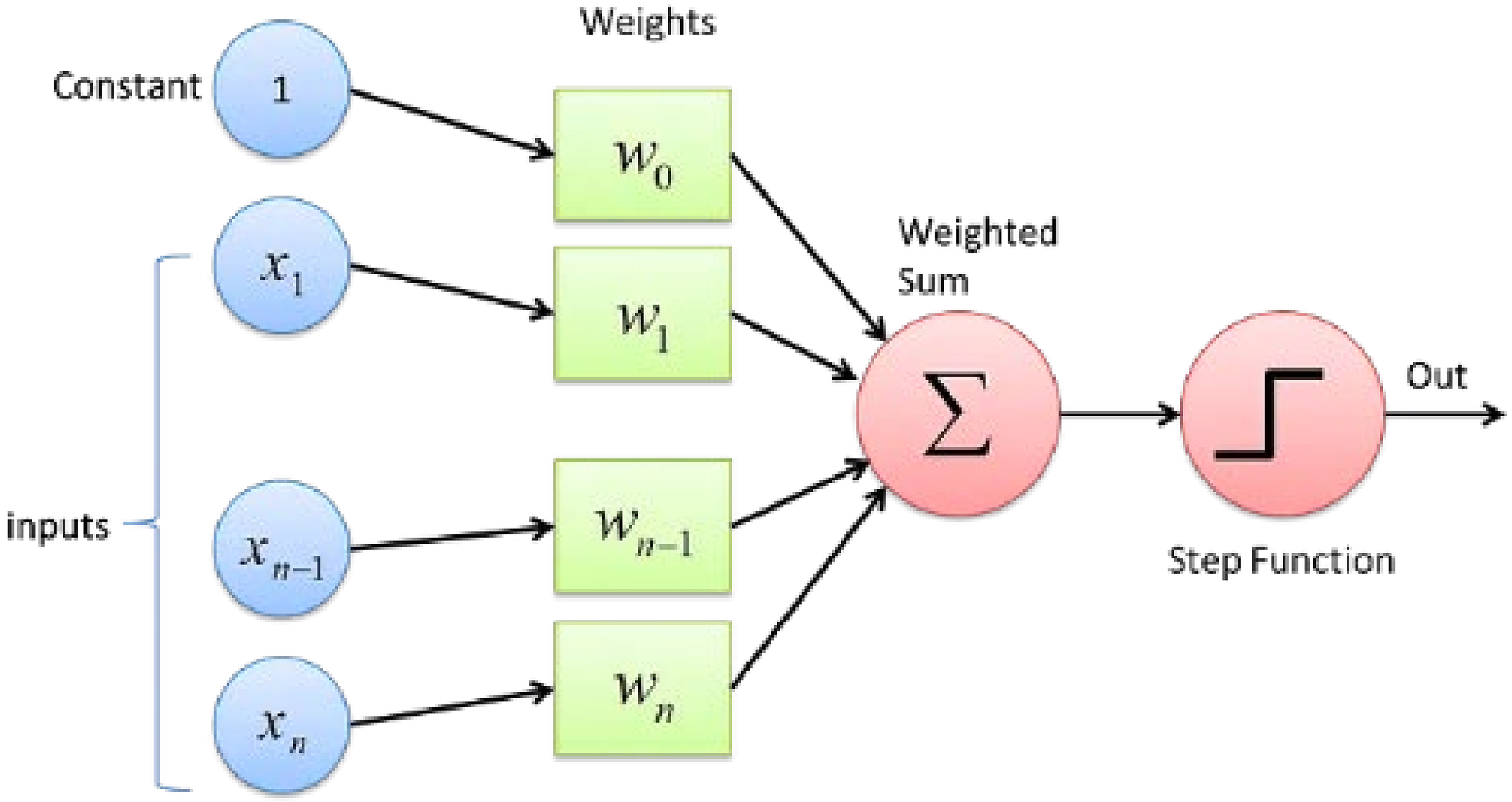
ARTIFICIAL NEURAL NETWORKS

- Artificial neural networks (ANNs) are a class of machine learning models inspired by the structure and function of the brain
- The first ANN was introduced in 1943 by Warren McCulloch (a neurophysiologist) and Walter Pitts (a mathematician and logician)
- In the 1940s it was known that the brain is built of neurons wired together
  - Each neuron has inputs (dendrites)
  - As well as outputs (axon)
  - When impulses coming in exceed a threshold, the neuron emits a pulse, which is taken as input to the next neuron

THE FIRST ANNS

- The network proposed by McCulloch and Pitts was a binary threshold network
  - Inputs can only be binary
  - Used for simple logical operations
- In 1958, Frank Rosenblatt proposed a more complex ANN called the **Perceptron**
  - Inputs are weighted sums of inputs
  - Binary output is based on an activation function
  - Has the ability to learn by adjusting weights according to its mistakes

THE PERCEPTRON



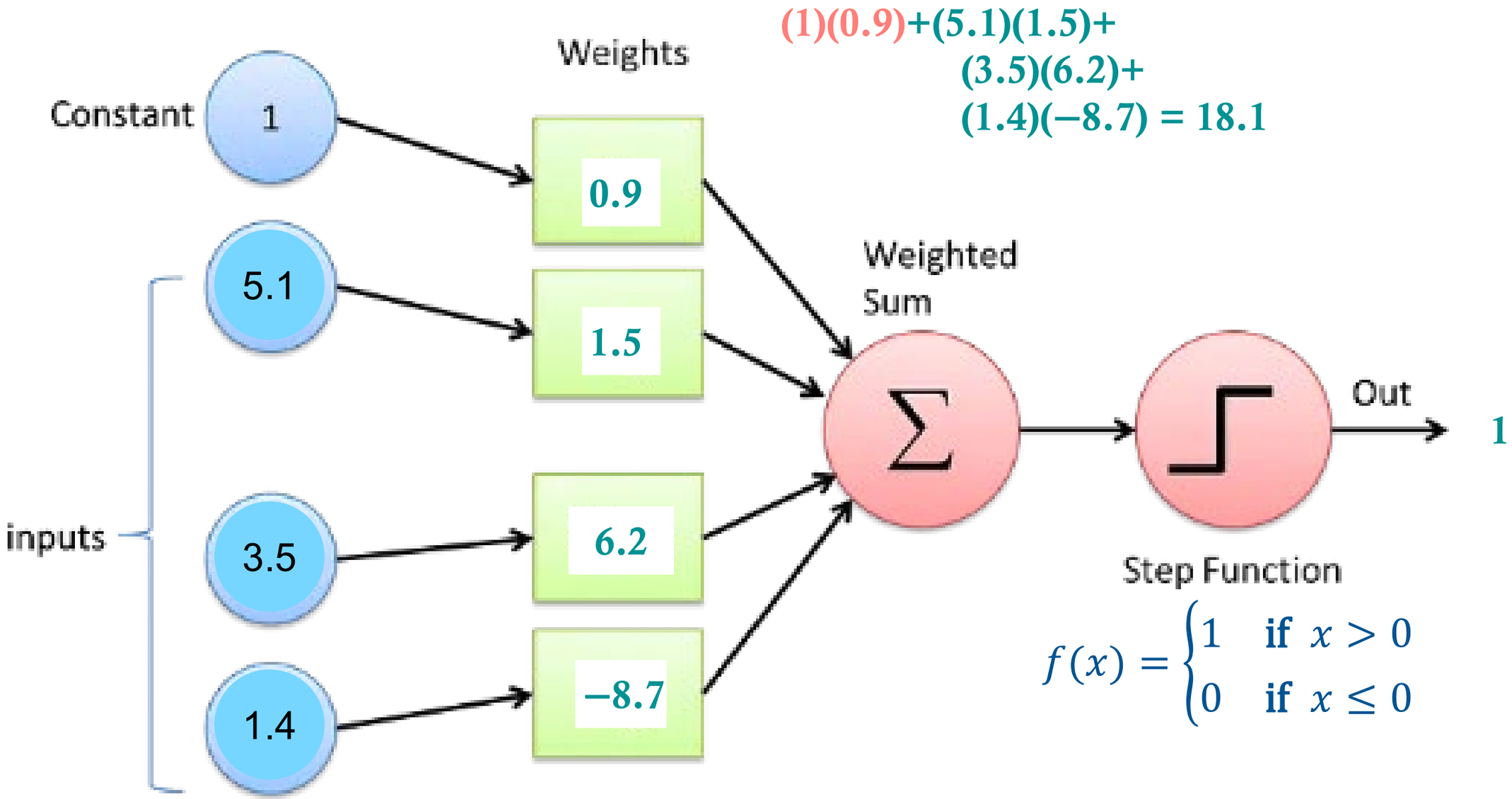
(1) Receive input

(2) Assign weights to the inputs

(3) Compute the weighted sum

(4) Generate output according to the activation function

THE PERCEPTRON



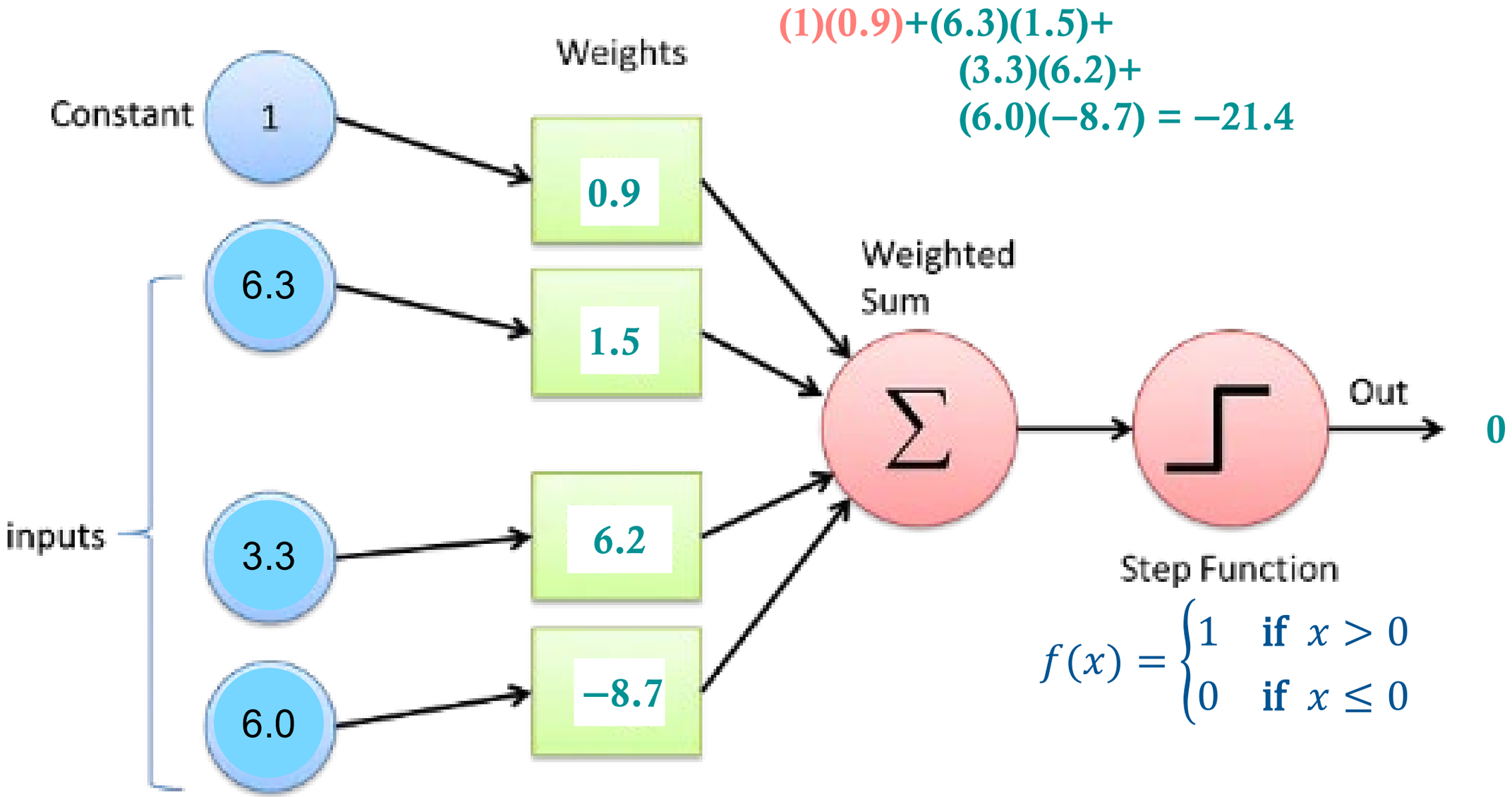
(1) Receive input

(2) Assign weights to the inputs

(3) Compute the weighted sum

(4) Generate output according to the activation function

THE PERCEPTRON



(1) Receive input

(2) Assign weights to the inputs

(3) Compute the weighted sum

(4) Generate output according to the activation function

TRAINING THE PERCEPTRON

- Training is the process of find the weights of the Perceptron
- Perceptron Algorithm:
    - Assign weights randomly
    - Compute the output
    - Compare the output to the true values
    - Adjust weights according to the error by:
        - increasing weights that were too low and
        - decreasing weights that were too high
    - Repeat

PERCEPTRON TRAINING ALGORITHM

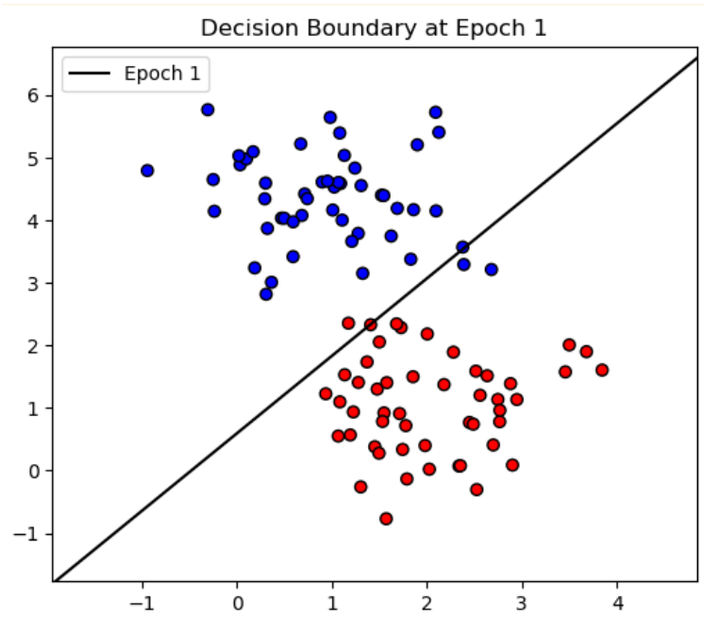
1. Initialize the weights (and bias) with small random values or to zero
2. For each epoch:
    a. For each training instance:
        i. Compute the weighted sum: $z = w \cdot x + b$
        ii. Apply the activation function: $\hat{y} = \begin{cases} 1, & \text{if } z > 0 \\ 0, & \text{otherwise} \end{cases}$
        iii. Update the weights if misclassified:

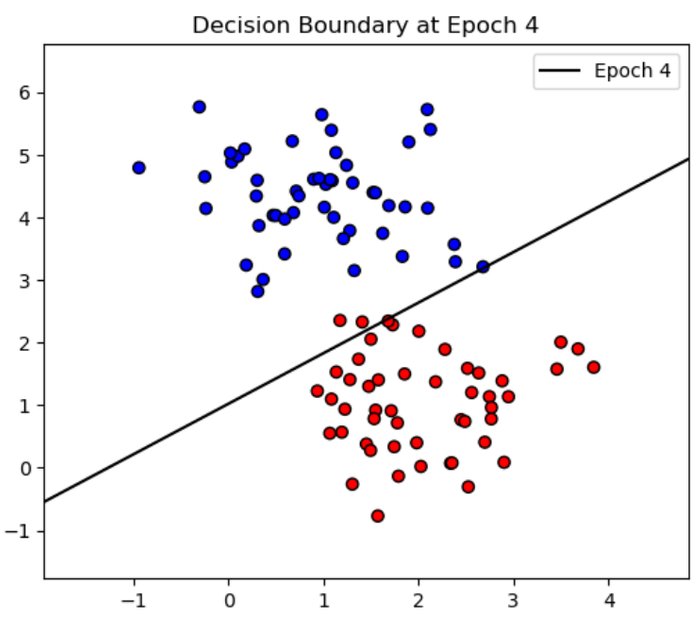
$$w \leftarrow w + \eta(y - \hat{y})x$$
$$b \leftarrow b + \eta(y - \hat{y})$$

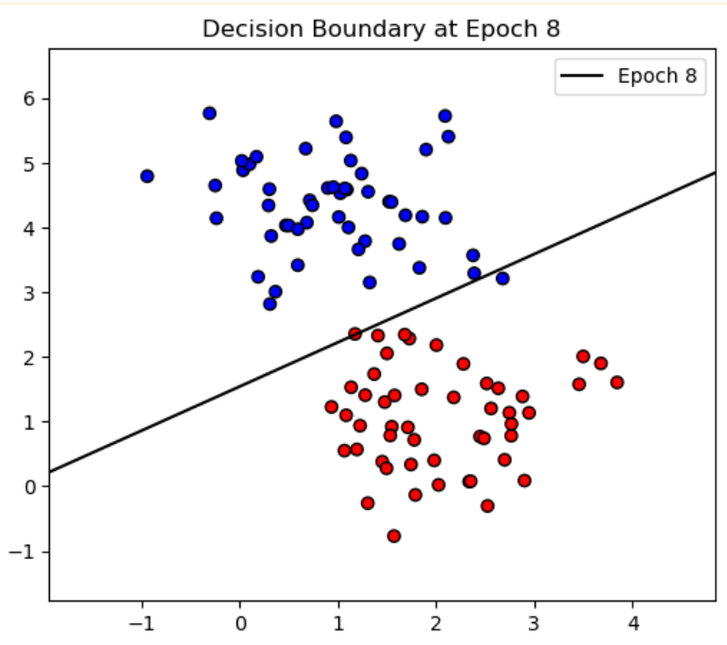
Note: Updates only happen when misclassified.

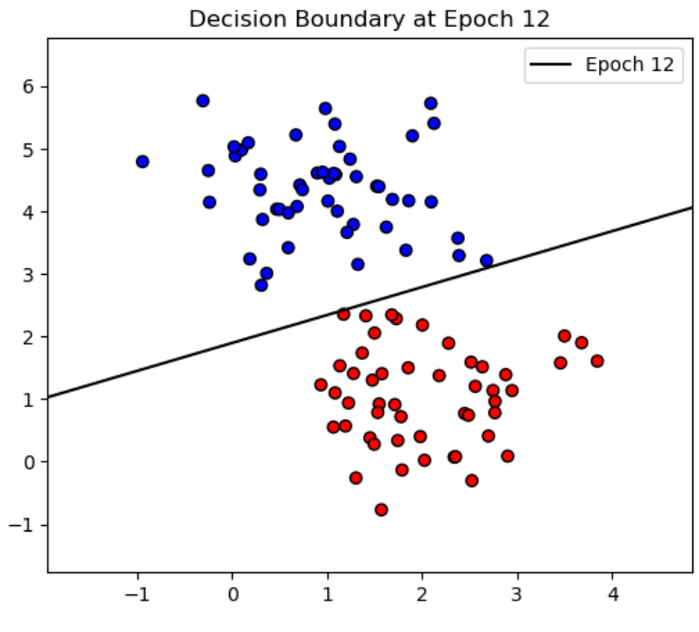
PERCEPTRON TRAINING ALGORITHM

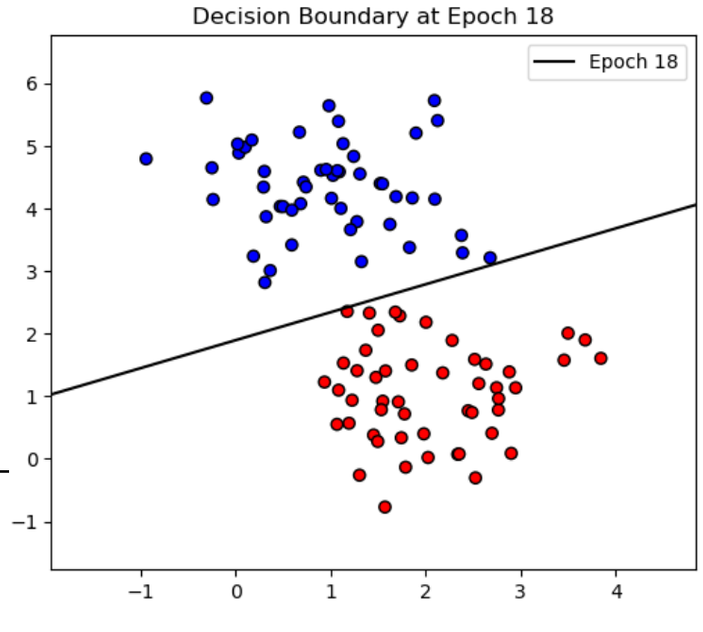
- If the classes are linearly separable, the perceptron algorithm will converge



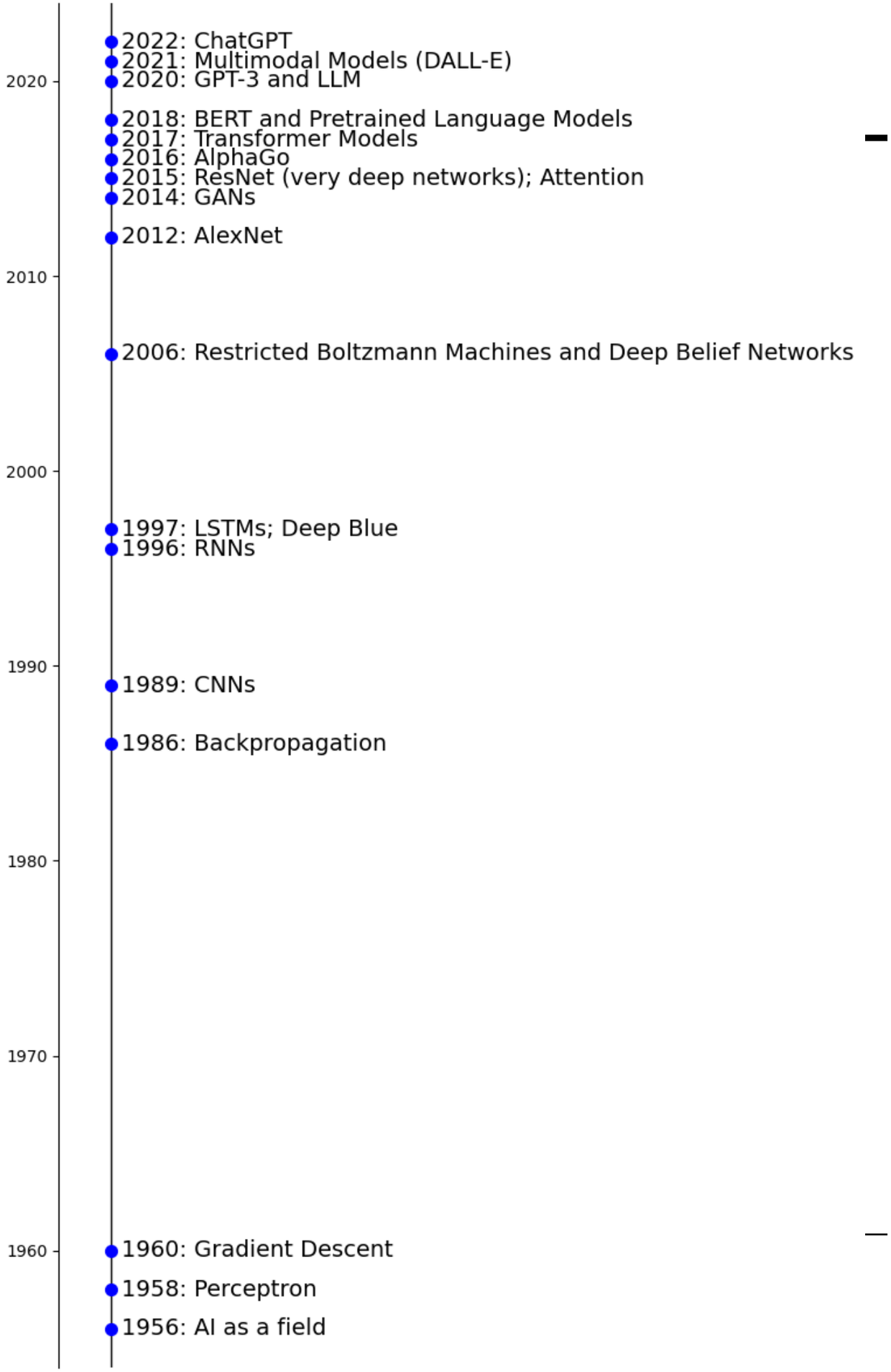




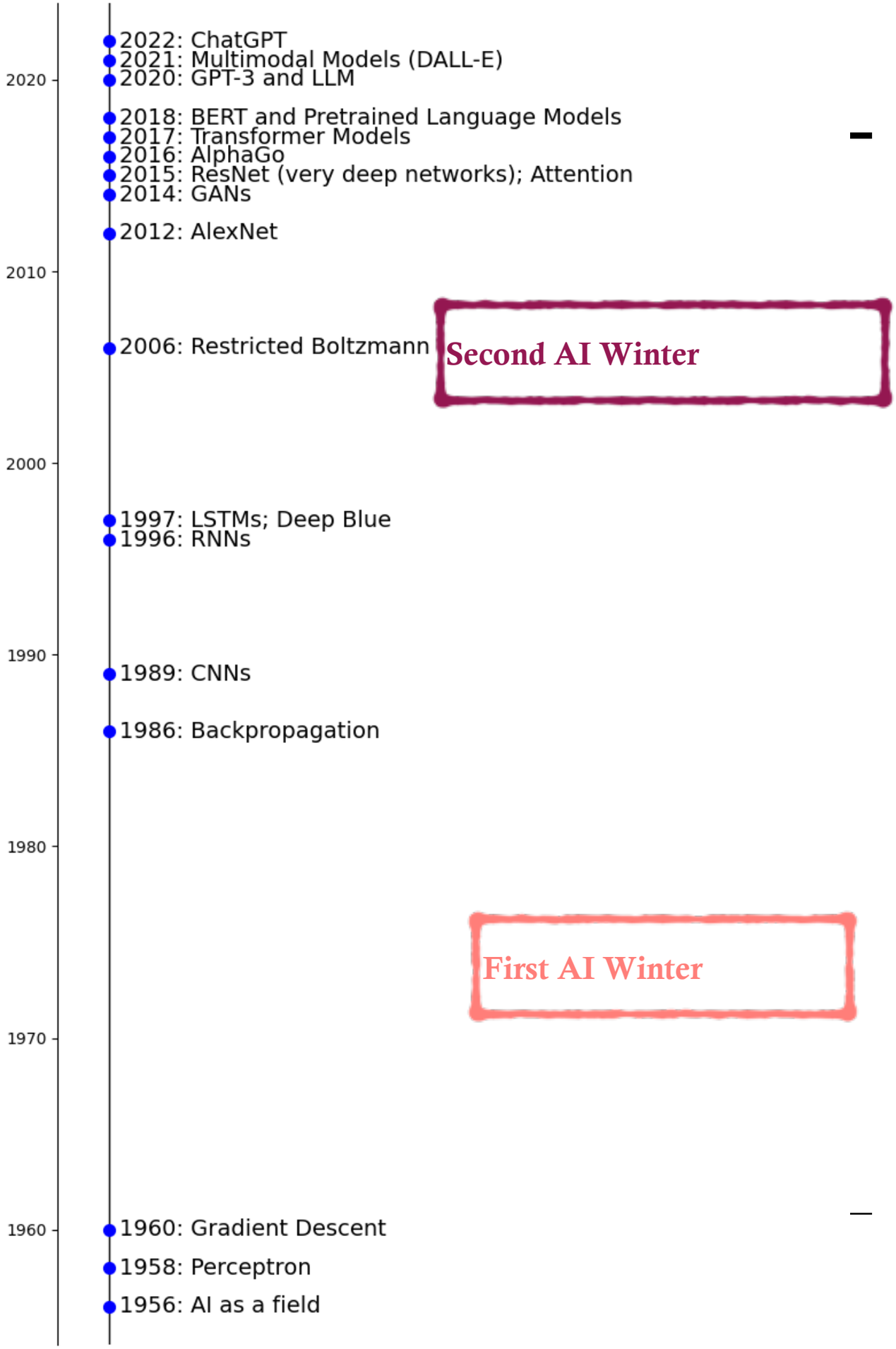




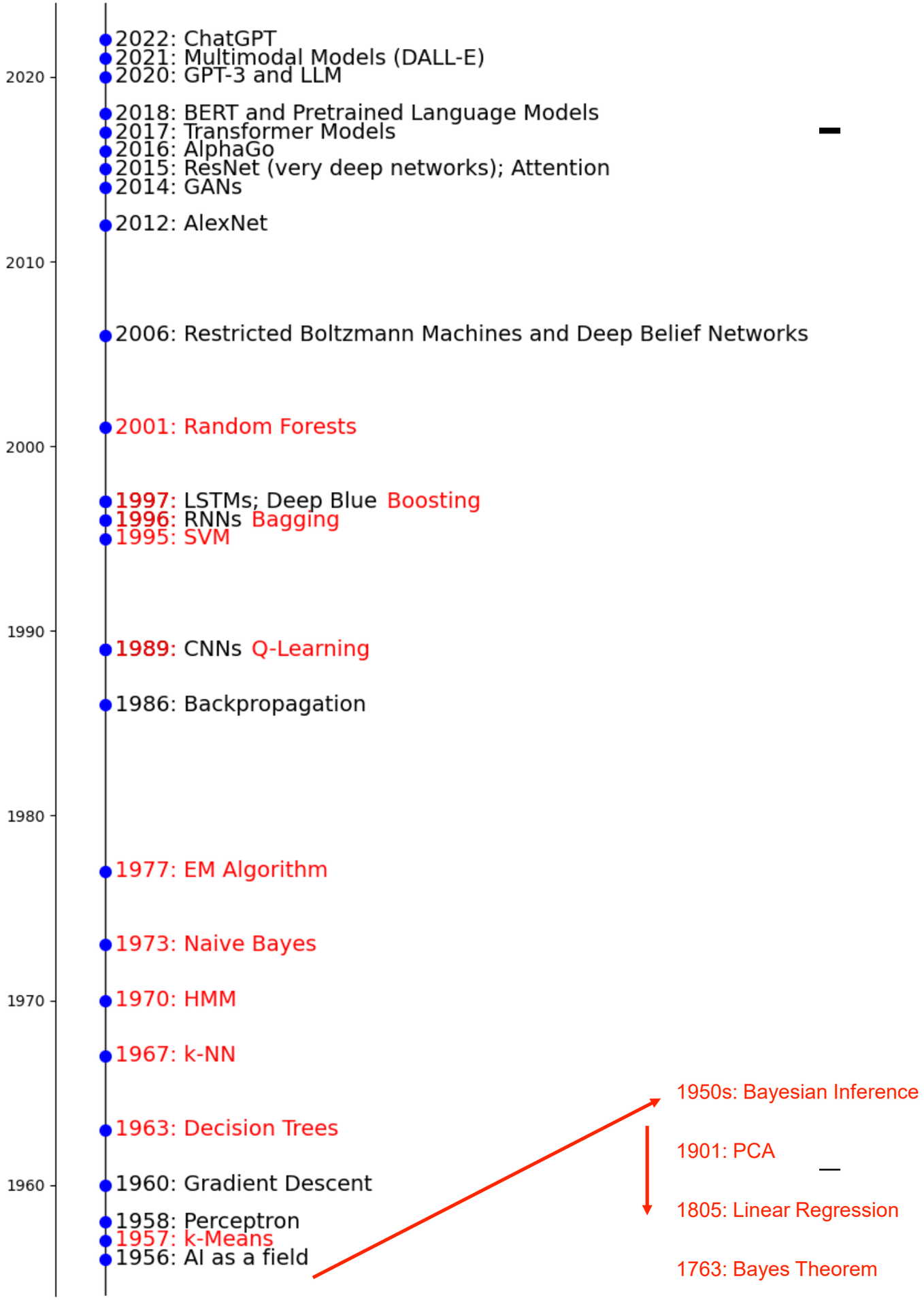
TIMELINE OF DEEP LEARNING MILESTONES



TIMELINE OF DEEP LEARNING MILESTONES



TIMELINE OF DEEP LEARNING MILESTONES

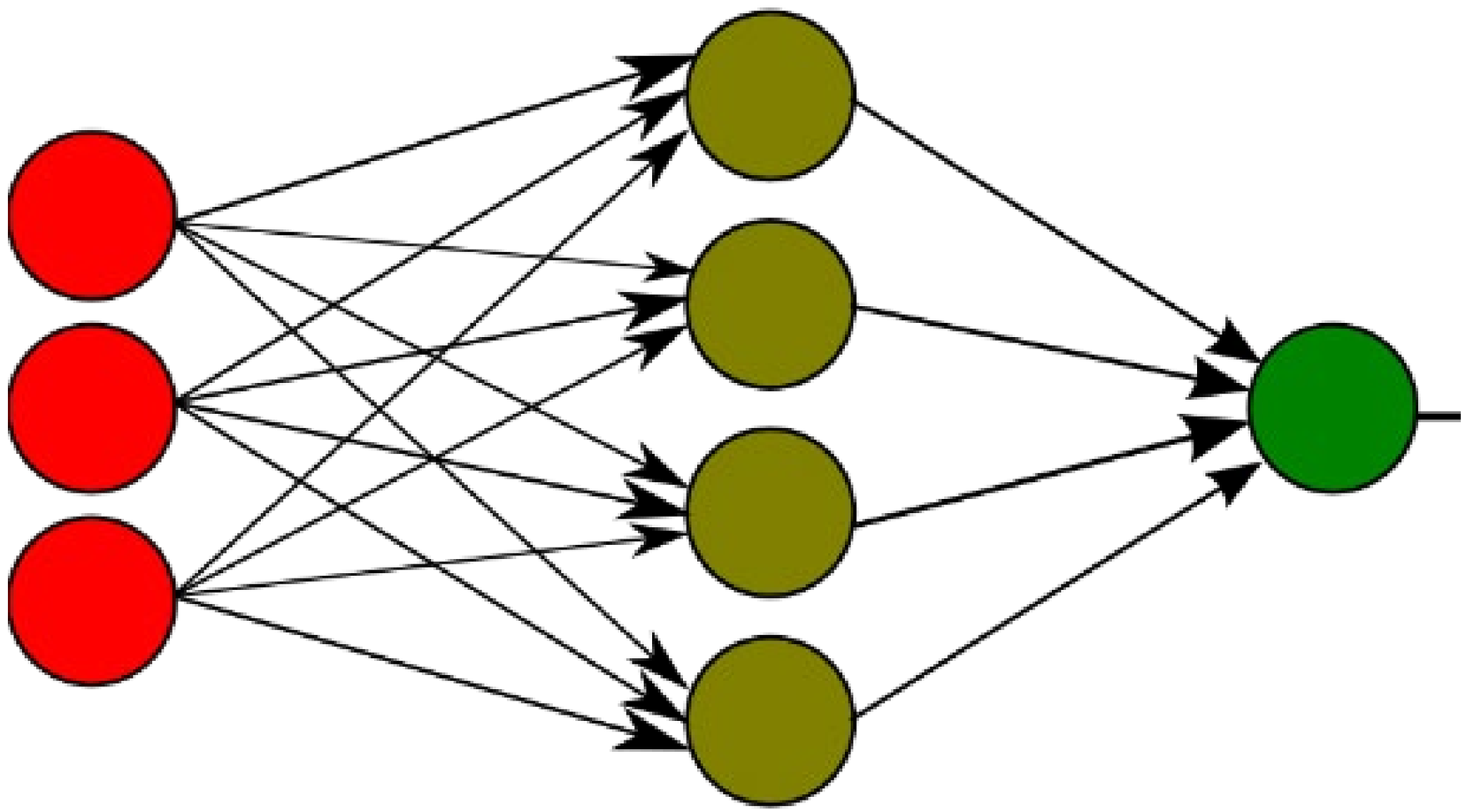


IMPROVEMENTS TO THE PERCEPTRON

- Nonlinear activation functions
  - The original step function activation is not able to capture complex nonlinear relationships
  - Nonlinear differentiable functions (such as the sigmoid or hyperbolic tangent were introduced
- Multiple layers
  - Hidden layers can capture complex, nonlinear relationships
  - Each layer can transform the input in sophisticated ways

MULTILAYER PERCEPTRON

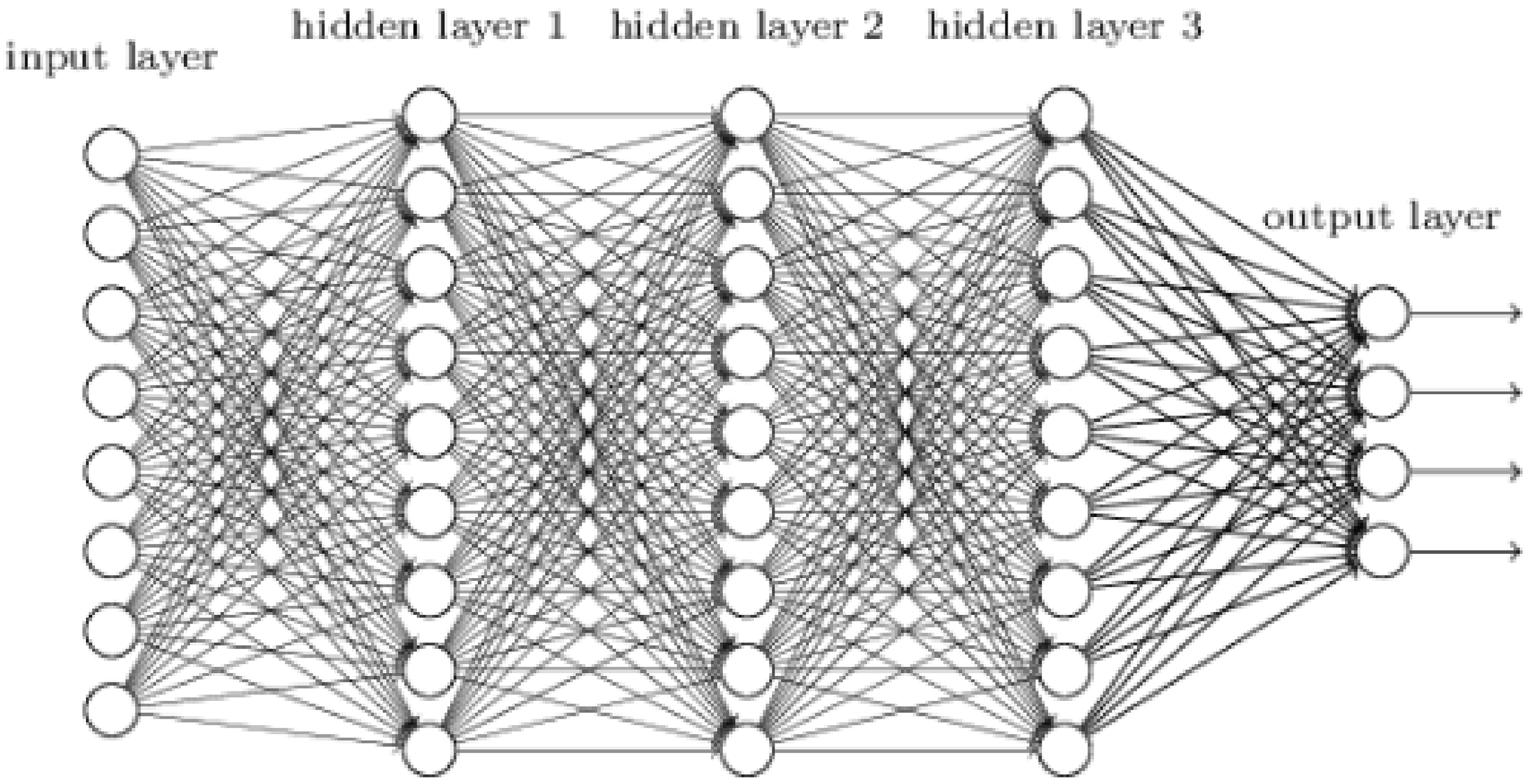
Also called a **fully connected feed-forward neural network**



Input Layer | Hidden Layer | Output Layer

MULTILAYER PERCEPTRON

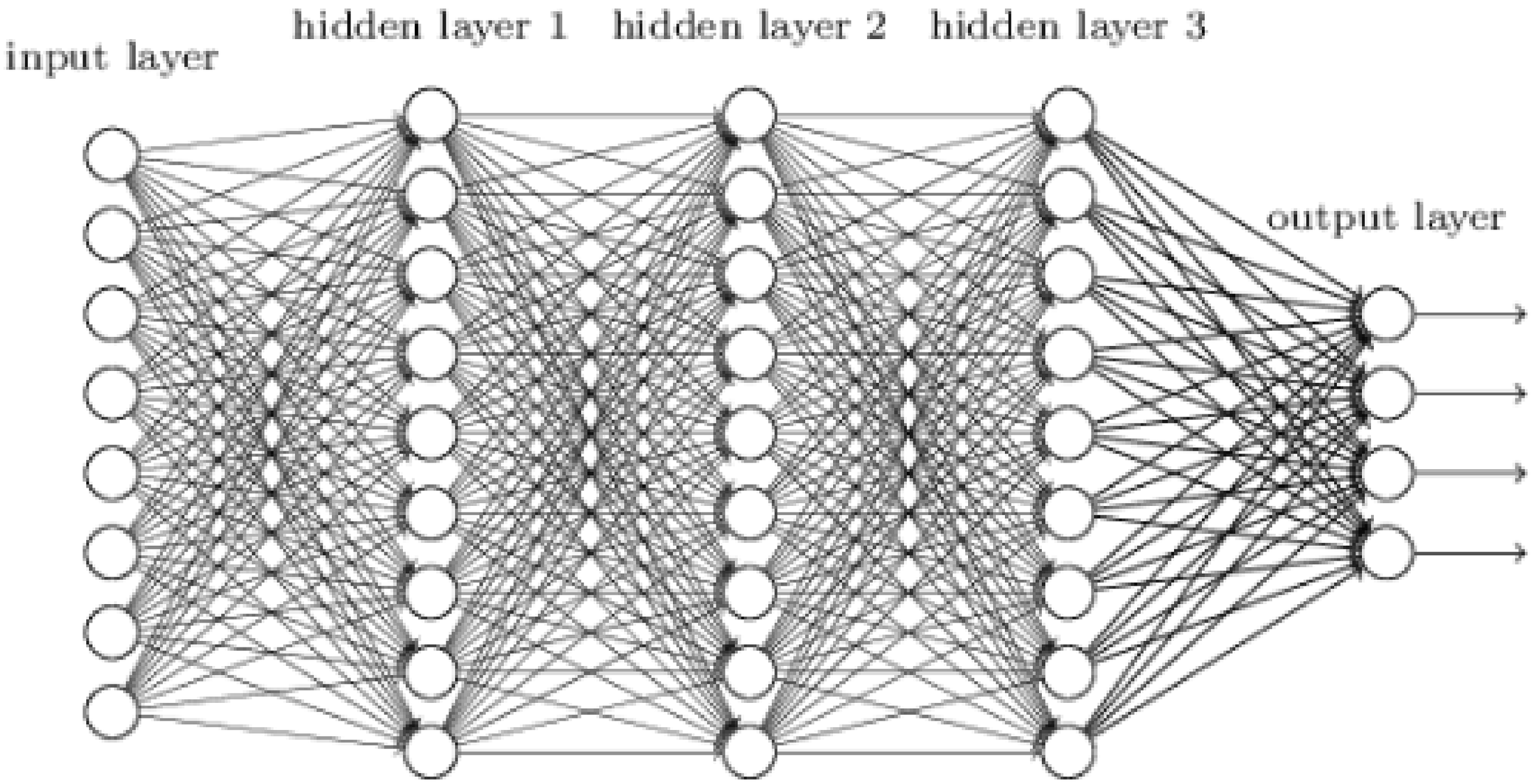
Also called a **fully connected feed-forward neural network**



PROBLEM

- How can we efficiently train the multilayer perceptron?
  - Adjust the weights at every level
  - In a computationally feasible way

MULTILAYER PERCEPTRON



Approximately, how many parameters are estimated?

NEW ANN EXCITEMENT

- In 1986, a paper showed how to train ANNs with "back-propagation"
  - Discovered independently several times earlier, but not popular until 1986
- Backpropagation efficiently computes the gradient of a loss function
  - Gradient for each weight computed **using the chain rule one layer at a time**
  - Iterates backward from the last layer **to avoid redundant calculations** of intermediate terms
- Weights are then adjusted using gradient descent or similar

CHAIN RULE: REVIEW

- **Single-variable (composition):**
  If $y = f\big(g(x)\big)$, then

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}f}{\mathrm{d}g} \cdot \frac{\mathrm{d}g}{\mathrm{d}x}$$

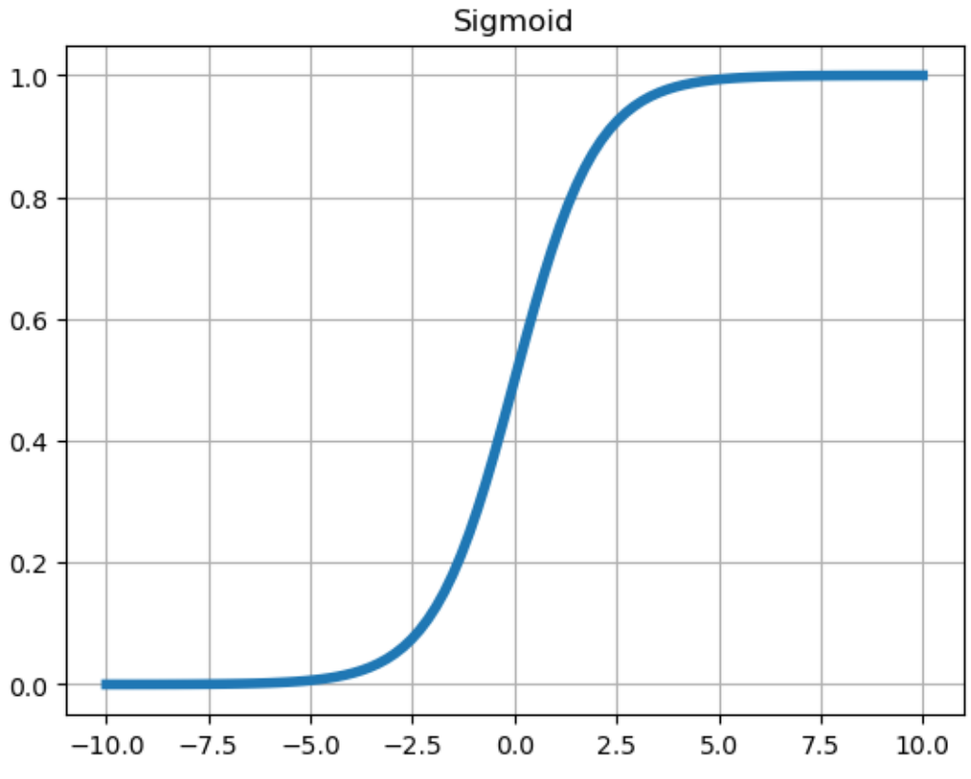
- **Multiple layers:**
  If $y = f(u),\ u = g(v),\ v = h(x)$, then

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\partial y}{\partial u}\frac{\partial u}{\partial v}\frac{\partial v}{\partial x}$$

NEW ACTIVATION FUNCTION

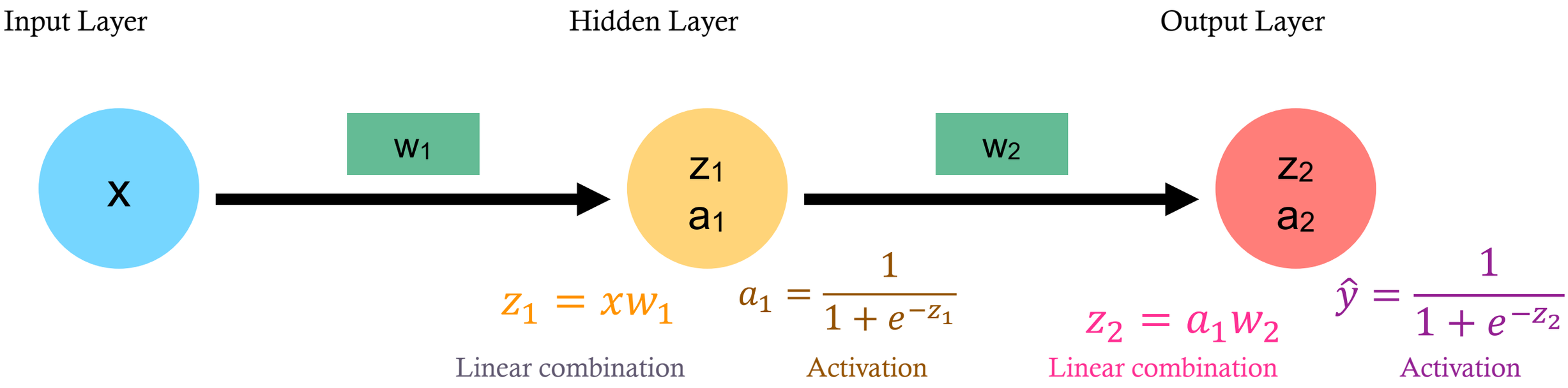
- In order for the partial derivatives to exist at every layer
  - The non-differentiable activation function (step) is changed for a differentiable activation function
  - The sigmoid (logistic) function was used in the original MLP

$$f(x) = \frac{1}{1 + e^{-x}}$$



UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:

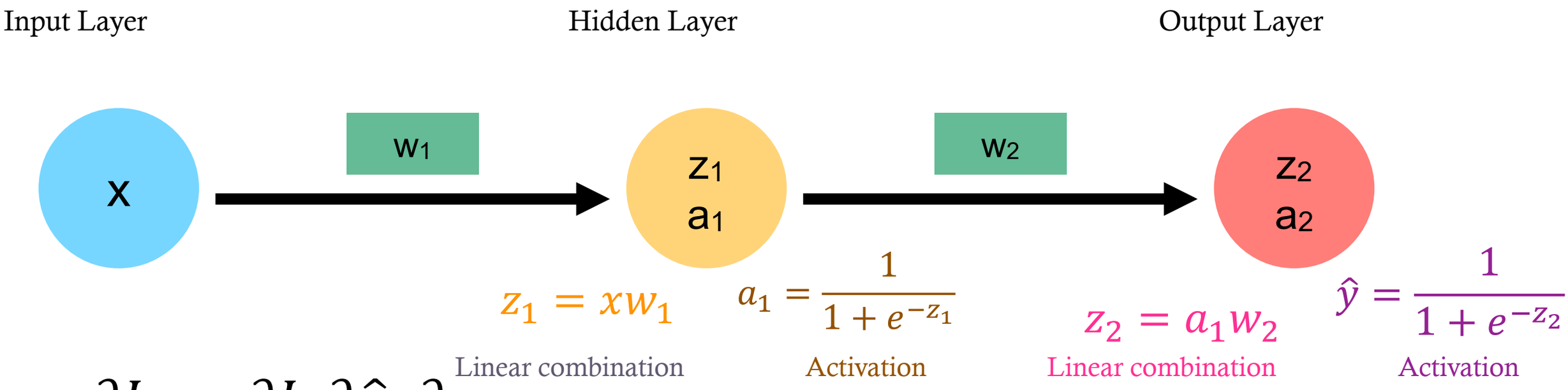


The goal is to compute the gradient of cost function *J* with respect to w1 and w2 in order find the direction of steepest descent

$$J(w) = \frac{1}{2}(\hat{y} - y)^2$$

UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:



$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial w_2}$$

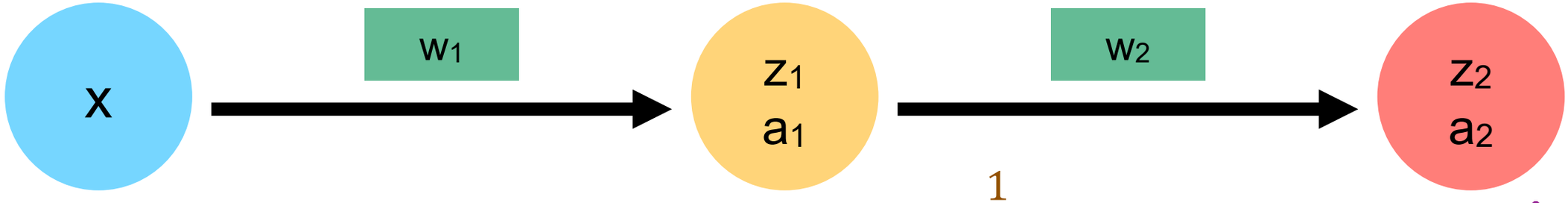
$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_1}$$

$$J(w) = \frac{1}{2}(\hat{y} - y)^2$$

UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:

Input Layer | Hidden Layer | Output Layer



$z_1 = xw_1$ Linear combination

$a_1 = \frac{1}{1+e^{-z_1}}$ Activation

$z_2 = a_1 w_2$ Linear combination

$\hat{y} = \frac{1}{1+e^{-z_2}}$ Activation

$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial w_2}$$

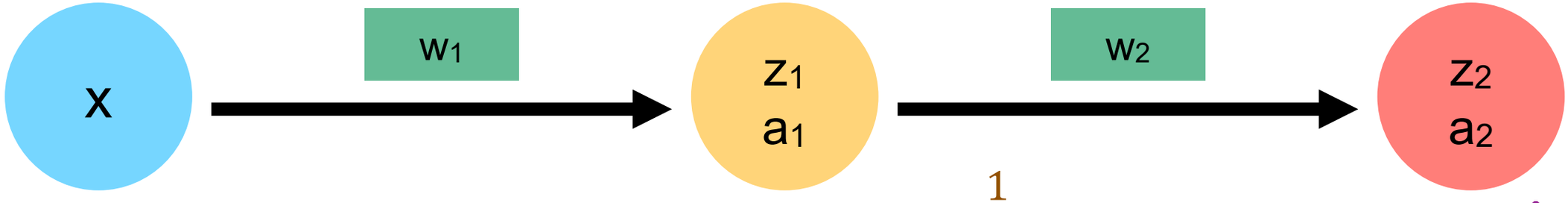
$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_1}$$

$$J(w) = \frac{1}{2}(\hat{y} - y)^2$$

UNDERSTANDING BACKPROPAGATION

- In order to develop intuition about the back propagation algorithm, consider a very simple neural network:

Input Layer | Hidden Layer | Output Layer



$z_1 = xw_1$ Linear combination

$a_1 = \frac{1}{1+e^{-z_1}}$ Activation

$z_2 = a_1 w_2$ Linear combination

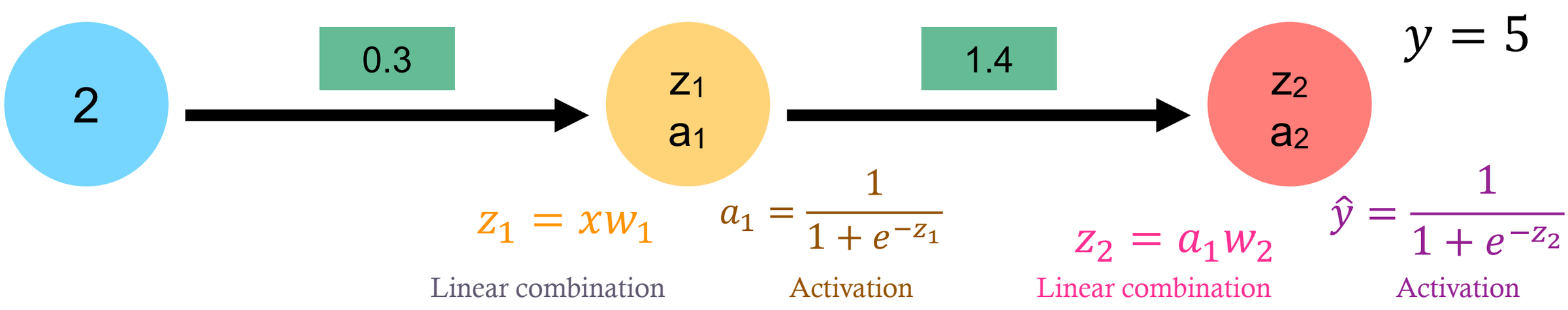
$\hat{y} = \frac{1}{1+e^{-z_2}}$ Activation

$$\frac{\partial J}{\partial w_2} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial w_2} = (\hat{y} - y)(1 - \hat{y})(\hat{y})(a_1)$$

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_2} \frac{\partial z_2}{\partial a_1} \frac{\partial a_1}{\partial z_1} \frac{\partial z_1}{\partial w_1} = (\hat{y} - y)(1 - \hat{y})(\hat{y}) w_2 (1 - a_1)(a_1) x$$

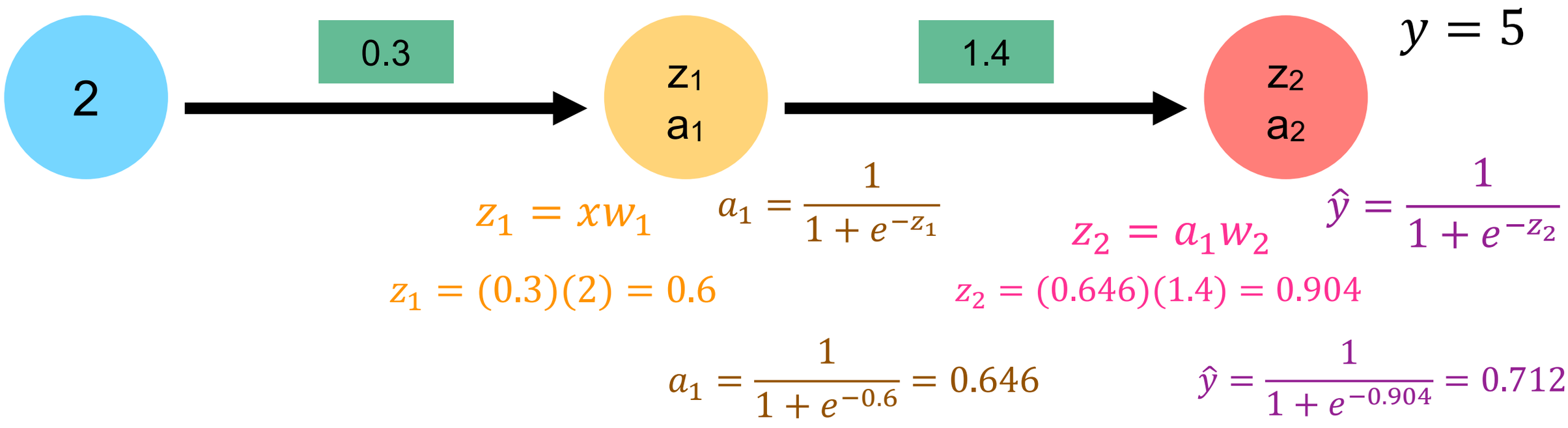
UNDERSTANDING BACKPROPAGATION

- Forward pass



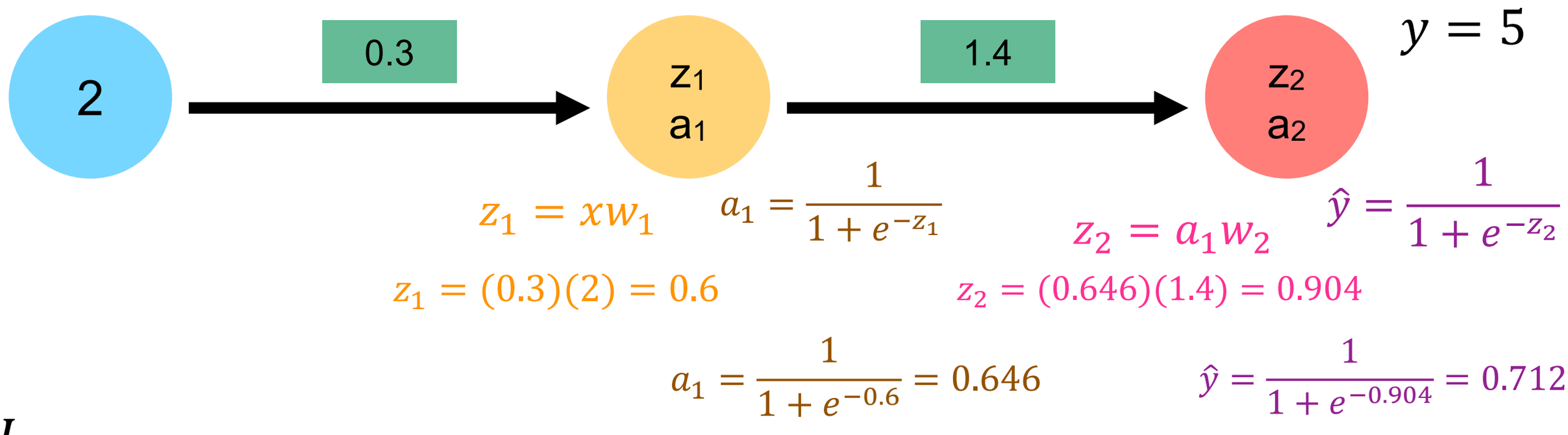
UNDERSTANDING BACKPROPAGATION

- Forward pass



UNDERSTANDING BACKPROPAGATION

- Back propagation

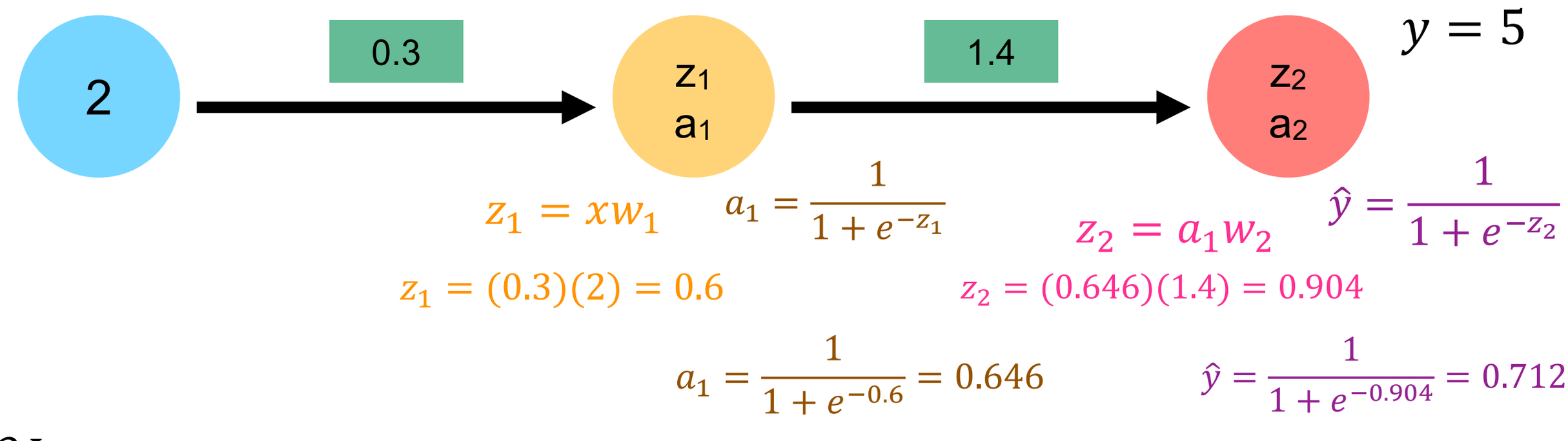


$$\frac{\partial J}{\partial w_2} = (y - \hat{y})(1 - \hat{y})(\hat{y})(a_1)$$

$$= (5 - 0.712)(1 - 0.712)(0.712)(0.646) = 0.568$$

UNDERSTANDING BACKPROPAGATION

- Back propagation

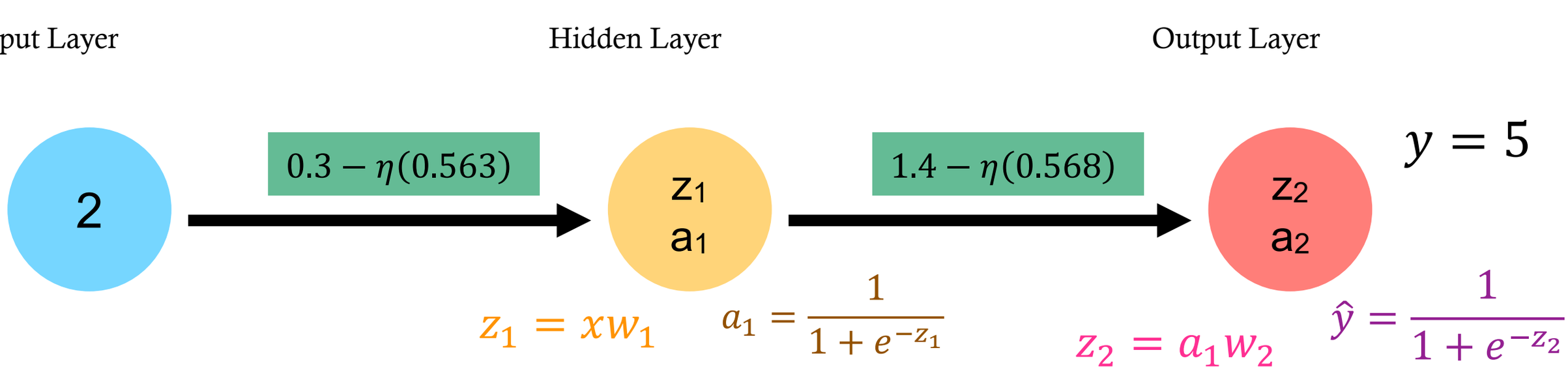


$$\frac{\partial J}{\partial w_1} = (y-\hat{y})(1-\hat{y})(\hat{y})w_2(1-a_1)(a_1)x$$

$$= (5-0.712)(1-0.712)(0.712)(1.4)(1-0.646)(0.646)(2) = 0.563$$

UNDERSTANDING BACKPROPAGATION

- Update and repeat



MORE PLACES TO UNDERSTAND

3Blue1Brown (YouTube Channel)

- Deep Learning
  - Chapter 1 (But what is a neural network? ~ 19min)
  - Chapter 2 (Gradient descent, how neural networks learn ~20min)
  - Chapter 3 (What is backpropagation really doing? ~ 13min)
  - Chapter 4 (Backpropagation calculus ~ 10min)

Jeremy Jordan Blog

- A thorough and easy-to-follow walkthrough of ANNs and backprop
- https://www.jeremyjordan.me/neural-networks-training/

OUTPUT LAYER

- Backpropagation shows how to train a network, but first the network must produce the right kind of output: the number of output neurons AND their activation function depend on the task
    - (what does the target look like?)
- ***Regression***
    - Usually one output layer, several activations functions possible (often the “identity” activation function)
    - For multivariate regression (multiple targets), one neuron is needed for each output dimension

OUTPUT LAYER

- The **number of neurons** in the output layer AND the **activation function** corresponds to the **desired output**
    - (what does the target look like?)
- ***Classification***
    - For binary classification, only one output neuron is needed using sigmoid activation function
    - For multiclass classification, one output neuron per class using the softmax activation function
        - Softmax is also possible for binary classification with two output neurons

SOFTMAX

- Converts raw scores (logits) into probabilities

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

- Outputs are in (0, 1)
- Sum to 1
- Emphasizes the largest logit (exponential scaling)
- Produces a pseudo-probability distribution
- Predicted class = $\arg\max_i \; softmax(z_i)$

ANOTHER DEEP PROBLEM

- Backpropagation with sigmoid units made training multilayer networks practical

- But the chain rule multiplies one **derivative per layer**, so gradients for early layers are **long products** of terms

- The sigmoid's derivative is never larger than 0.25. What happens to these products as depth grows?

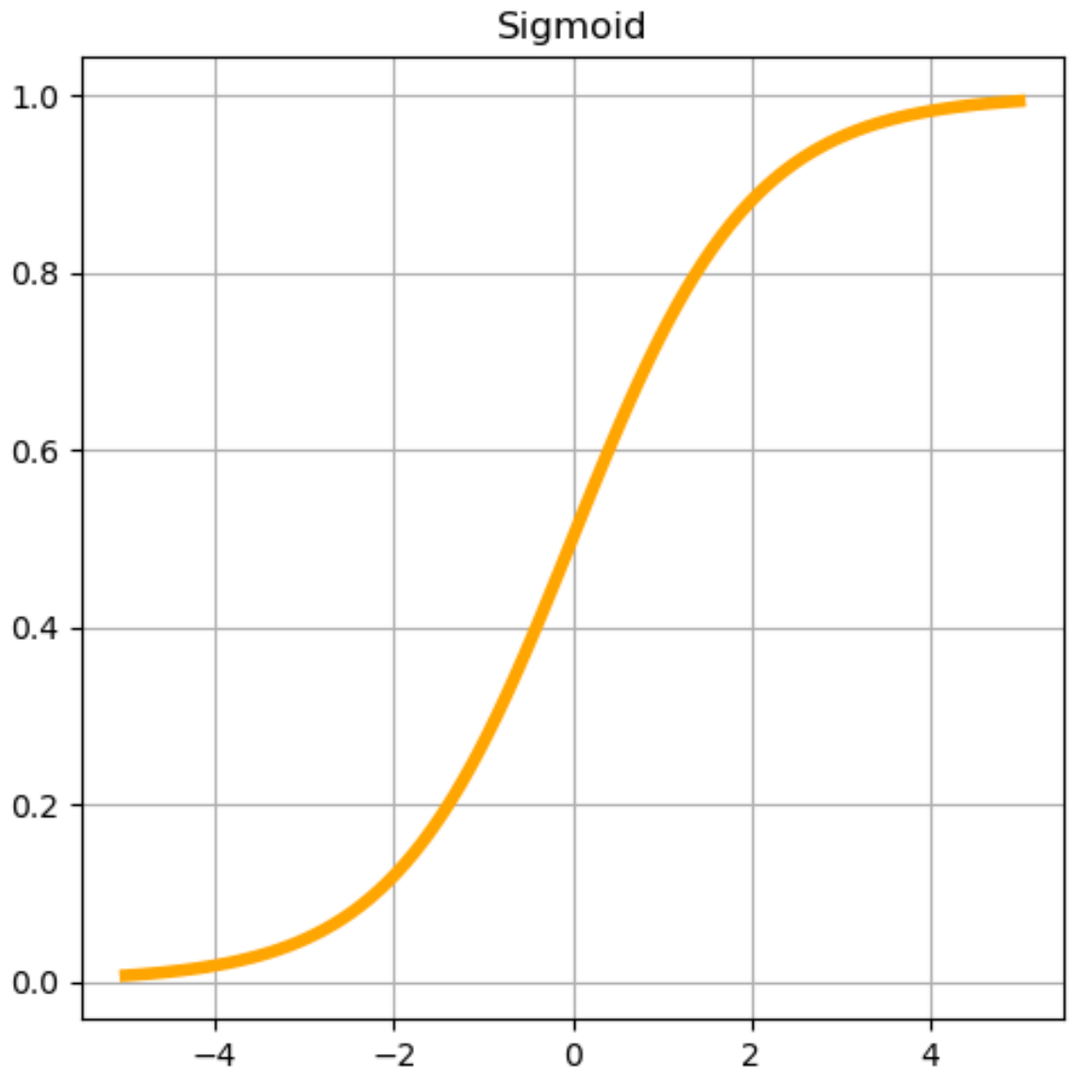
VANISHING GRADIENT PROBLEM

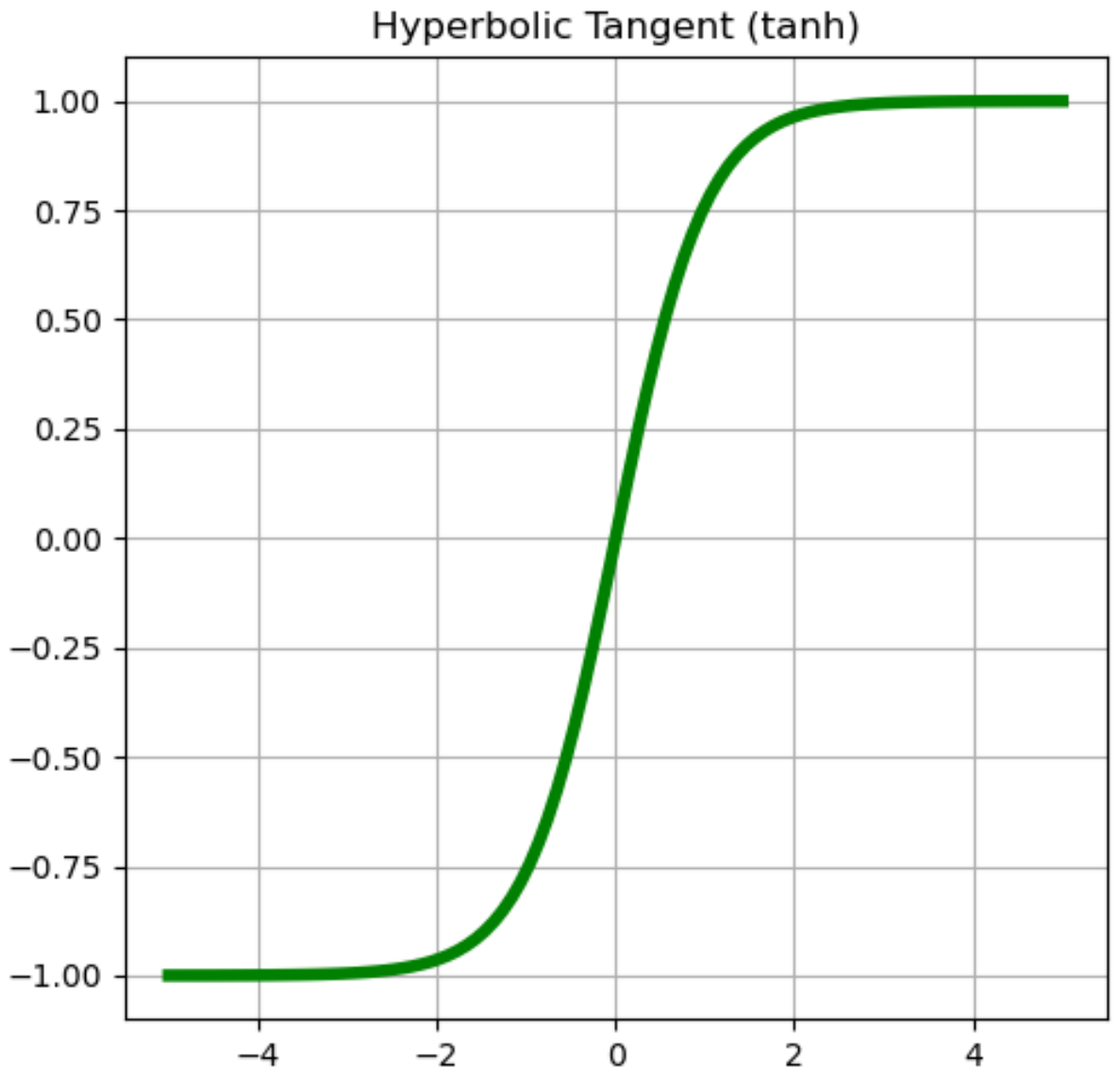
- When the derivative of the activation function is less than one, multiplying small numbers repeatedly causes the gradient to become very small

- This leads to **slow converges** and **poor performance**

- The opposite problem can also occur (exploding gradients)

SATURATING ACTIVATION FUNCTIONS





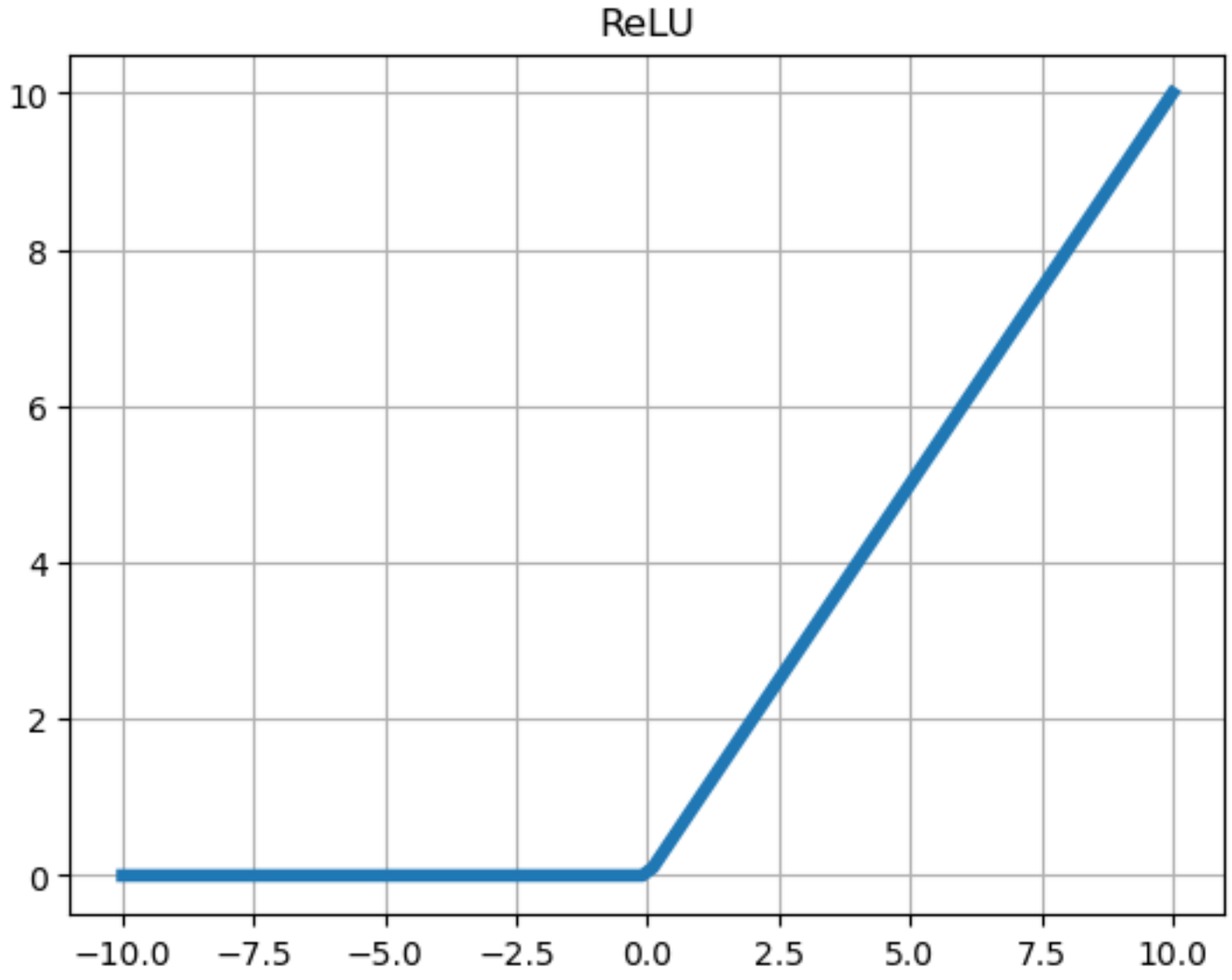
$$f(x) = \frac{1}{1 + e^{-x}}$$

$$f(x) = \tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

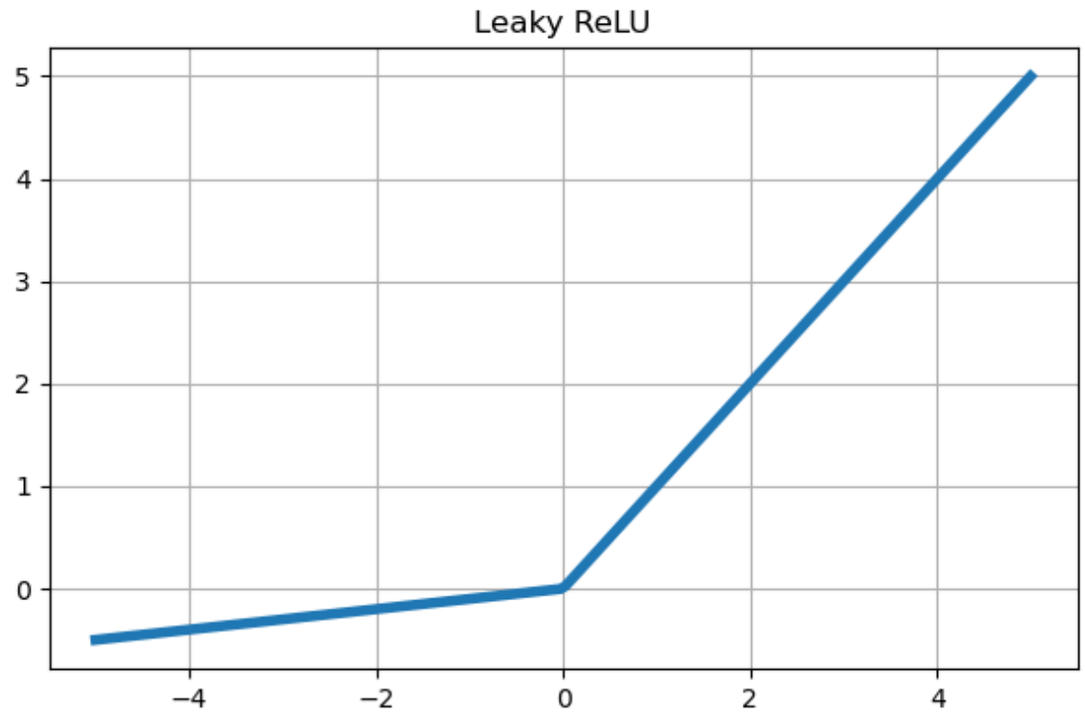
SOLUTIONS FOR UNSTABLE GRADIENTS

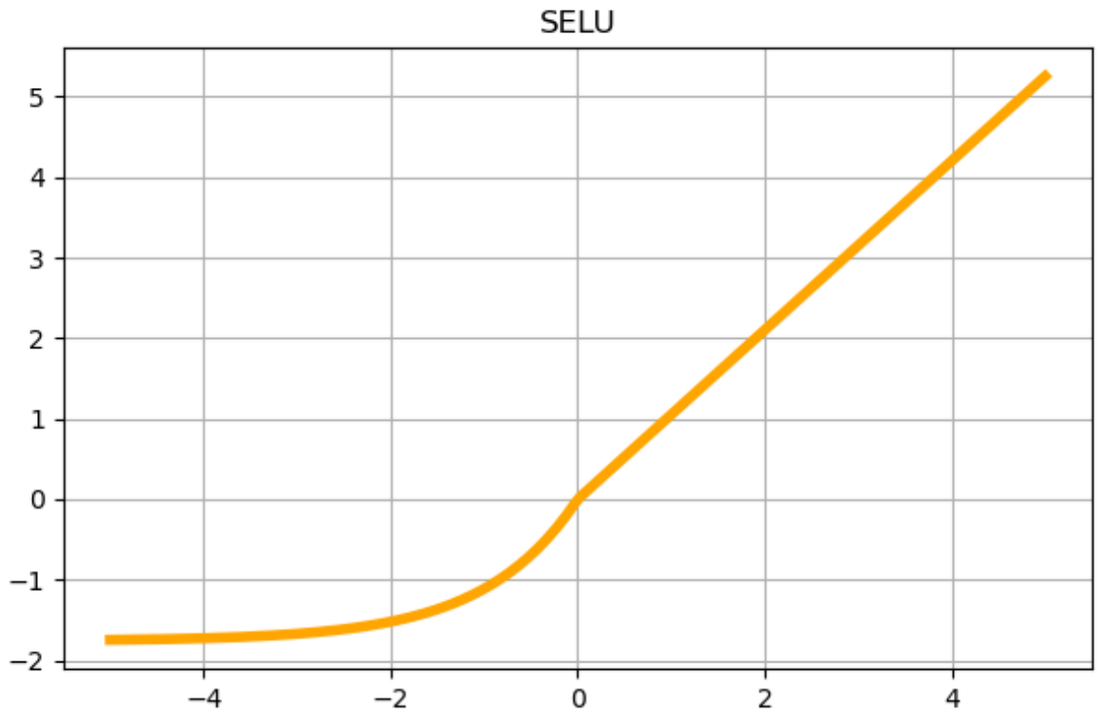
- **Activation functions**
  - Designed to alleviate this problem
  - ReLU and its variants (next two slides)
- **Weight initialization**
  - Sets weights to values that reduce the risk of unstable gradients at the start
- **Normalization**
  - Techniques such as batch normalization maintain stability by normalizing the inputs to each layer
- **Network architectures**
  - Some architectures are designed to help mitigate this issue

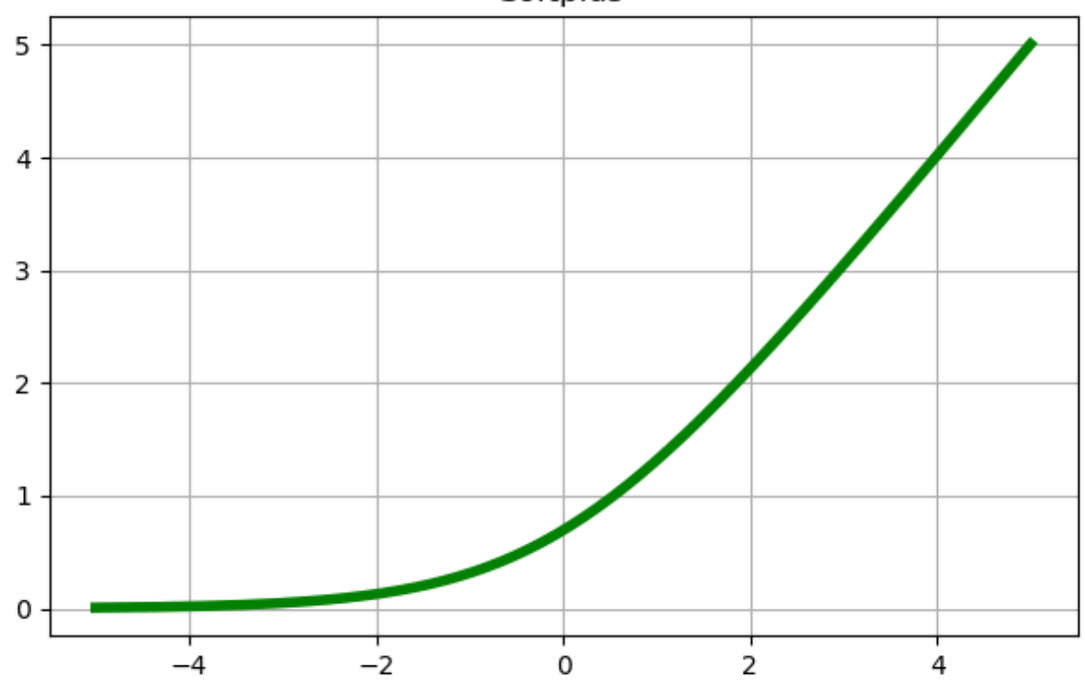
RECTIFIED LINEAR UNIT (RELU)

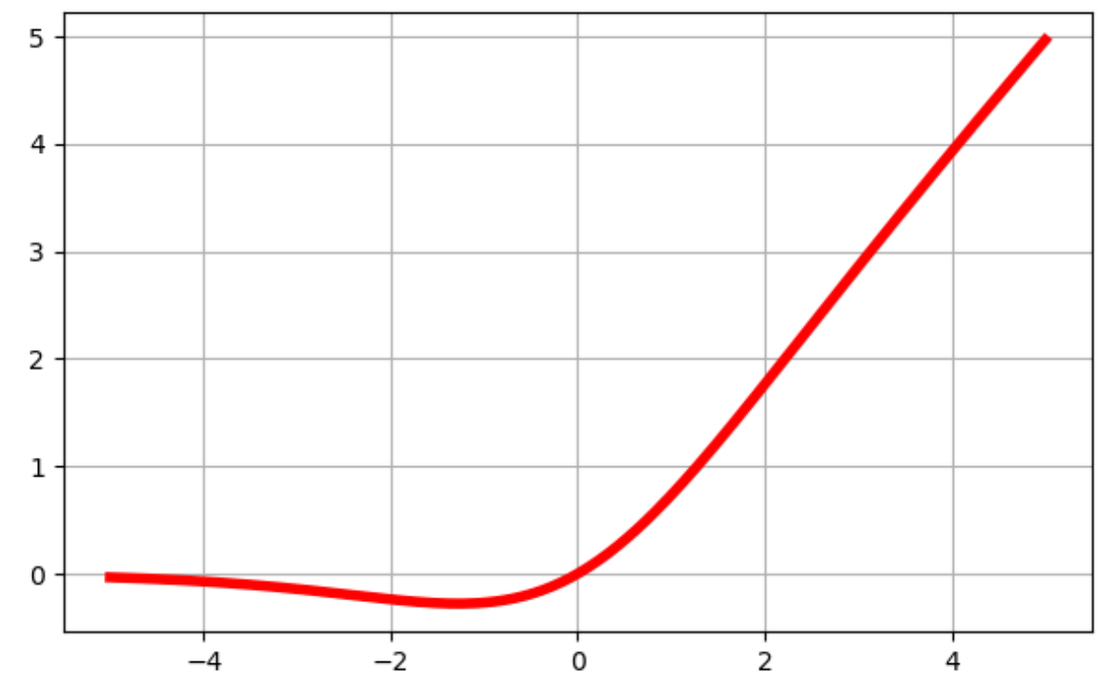


RELU VARIANTS (NOT AN EXHAUSTIVE LIST)









WEIGHT INITIALIZATION

- **Why it matters**
  - Poor starting weights make signals and gradients shrink or explode layer by layer
- **Break symmetry**
  - Zero or identical weights make every neuron in a layer learn the same thing, so use small random values
- **Keep variance stable**
  - Scale initial weights by layer size so activation and gradient variance stay roughly constant
- **Common schemes**
  - Glorot (Xavier) for sigmoid/tanh: variance 1 / fan_avg
  - He for ReLU: variance 2 / fan_in (ReLU zeroes half its inputs)
  - See pages 359-360 HOML

BATCH NORMALIZATION

- **The idea**
  - Normalize each layer's inputs using the mean and standard deviation of the current mini-batch

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma^2{}_B + \varepsilon}}, \qquad y = \gamma\hat{x} + \beta$$

- **Learned scale and shift**
  - γ and β let the network undo the normalization where that works better
- **Why it helps**
  - Reduces unstable gradients, allows higher learning rates, and adds a mild regularizing effect
- **At inference**
  - Uses running averages of the mean and standard deviation collected during training

TUNING A DNN

- Architecture
  - Number of hidden layers
  - Number of neurons per hidden layer
- Optimization
  - Learning rate
  - Optimizer
  - Batch size / epochs
- Generalization
  - Regularization
- Stability (covered in the unstable-gradient fixes)
  - Activation function / initialization / normalization

LEARNING RATE

- Step size of each update
  - Too high: may diverge; too low: slow or stuck
- Rules of thumb
  - Adam ≈ 1e-3; SGD ≈ 1e-2 to 1e-1; search on a log scale
- Learning rate range test
  - Grow rate exponentially; pick near steepest loss drop

SCHEDULES & ADAPTIVE RATES

- Learning rate schedules
  - Step, exponential, cosine decay, 1cycle, warmup
- Reduce on plateau
  - Cut rate by 2–10× when validation loss stalls
- Adaptive optimizers
  - AdaGrad, RMSProp, Adam scale rates per parameter
- Still tune the base rate
  - Adaptive methods reduce, not remove, the need

OPTIMIZER

- A single fixed learning rate is hard to get right, so variations of gradient descent were developed to be faster and more stable
  - Adam: Adaptive Moment Estimation
  - Requires minimal memory
  - Computationally efficient
  - Suited for large problem (large n and/or large p)
  - Effective for noise or sparse gradients

OPTIMIZER: OTHER OPTIONS

- SGD with momentum
  - Smooths updates; often generalizes well
- RMSProp / AdaGrad
  - Per-parameter rates; AdaGrad's rate decays too fast
- AdamW / Nadam
  - Adam with decoupled weight decay / Nesterov momentum
- Rule of thumb
  - Start with Adam; try tuned SGD+momentum if generalization lags

BATCH SIZE & NUMBER OF EPOCHS

- **Batch Size:** Number of training examples used in one iteration of model training
  - Model weights are updated after computing the gradient over batches
  - Large batch sizes are processed more efficiently but can lead to instabilities (see pg 352 HOML)
- **Epoch:** One full passthrough of the entire training data
  - Each epoch consists of the number of iterations needed to use all training examples once
  - Too many epochs can lead to overfitting and too few can lead to underfitting

REGULARIZATION: REDUCING OVERFITTING

- **L1 and L2 regularization**
    - Add a penalty to cost function and prevent weights from becoming too large
- **Dropout**
    - At every step, every neuron has probability p of being ignored
- **Early stopping**
    - Stop training if test error begins to go up
- **Data augmentation**
    - Artificially add more data (works especially well for CNNs)

Let’s take a look at the TensorFlow playground

(https://playground.tensorflow.org)

Try varying depth, activation, learning rate, and regularization